

Preparcial programación

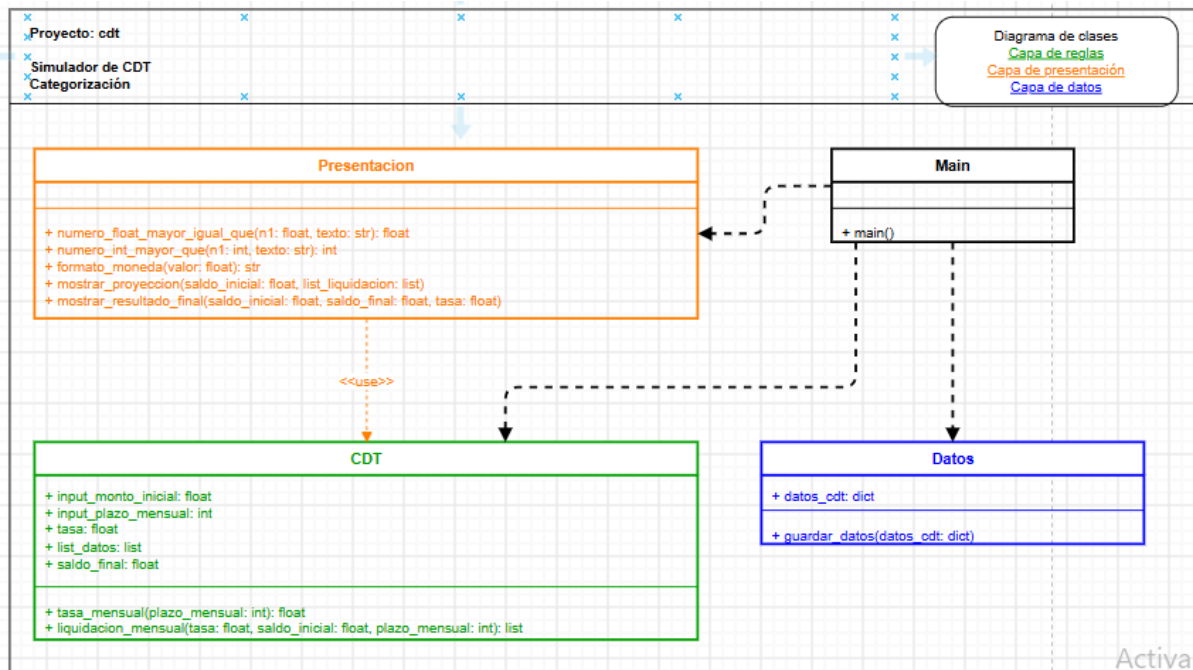
Fredy Omar Avila Triana

Codigo:01251151005

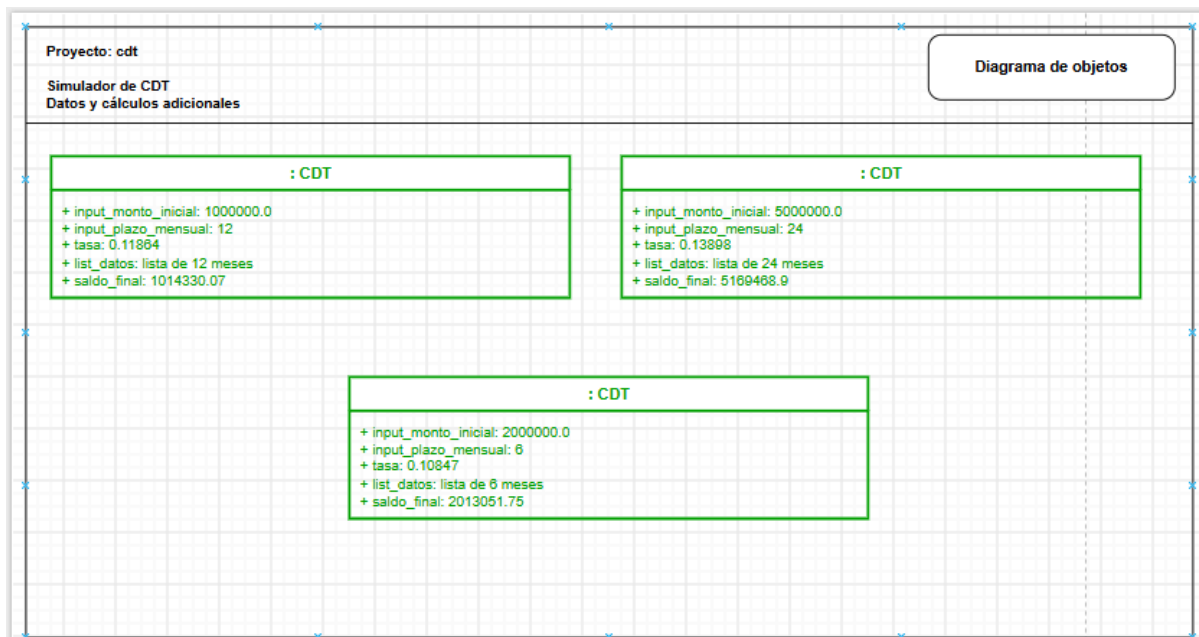
Andrés Felipe Castilla Caselles

Codigo:01251151011

1 Diagrama de clases:

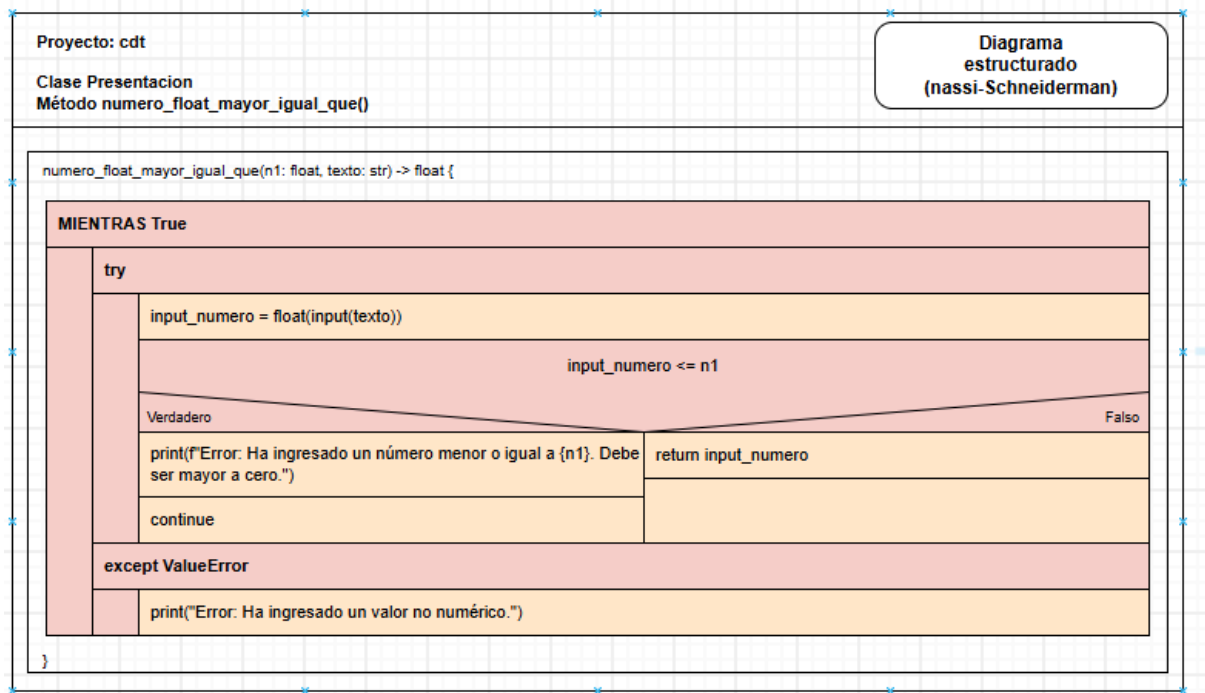


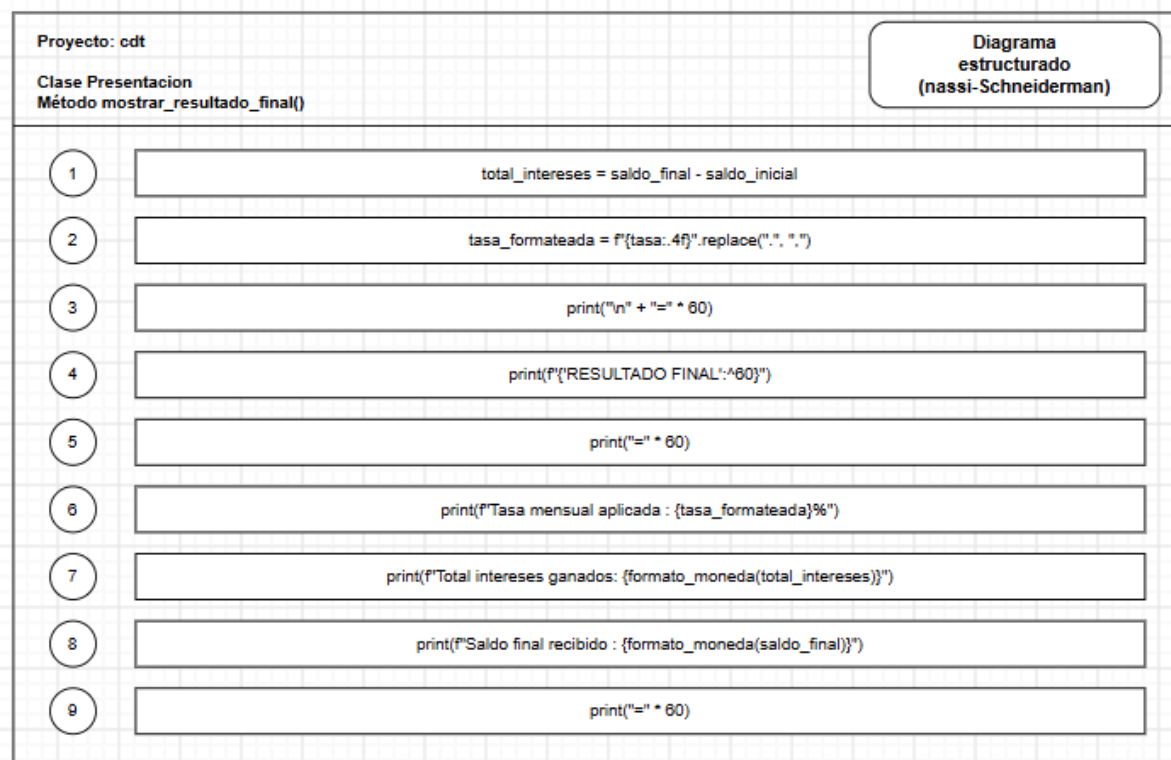
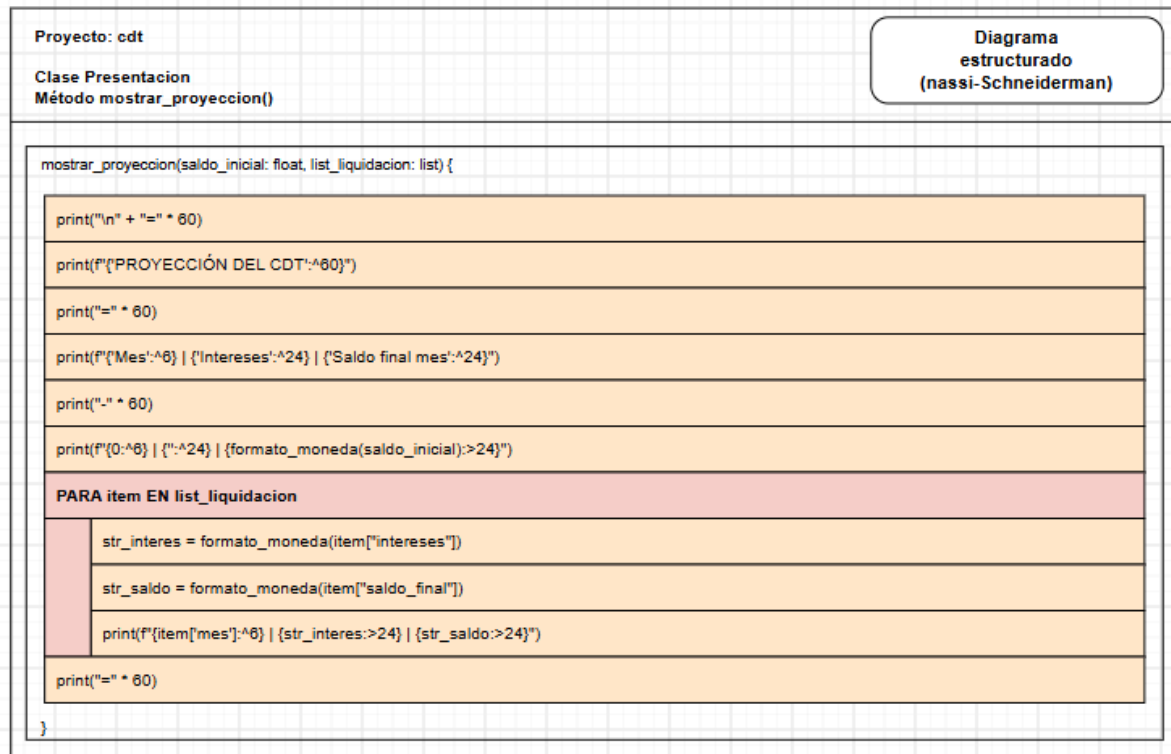
2 Diagrama de objetos:



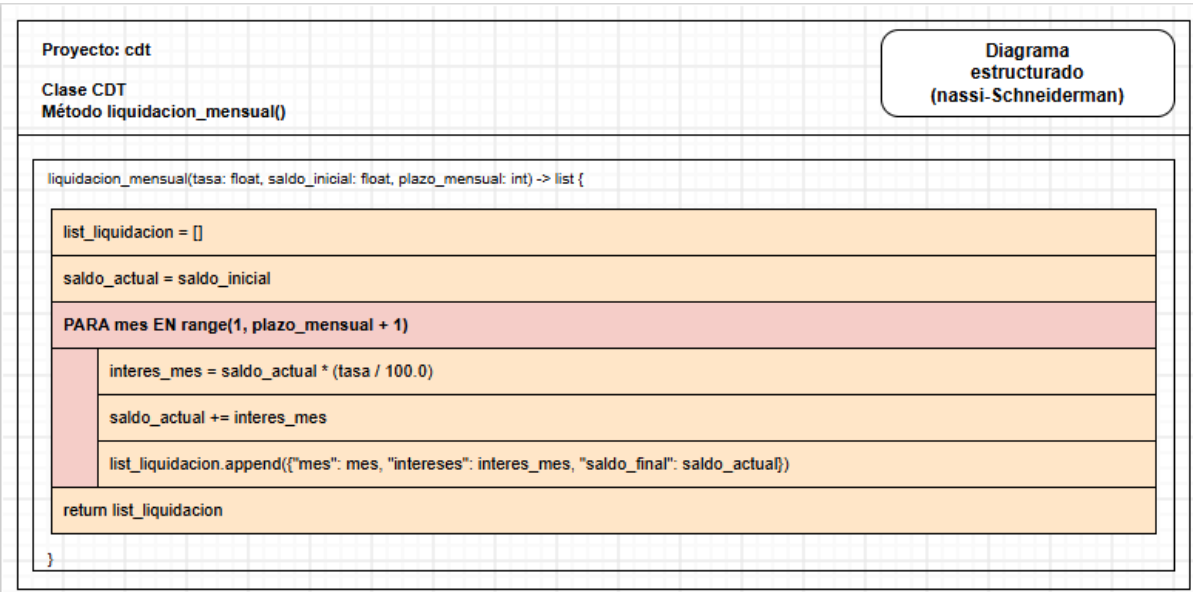
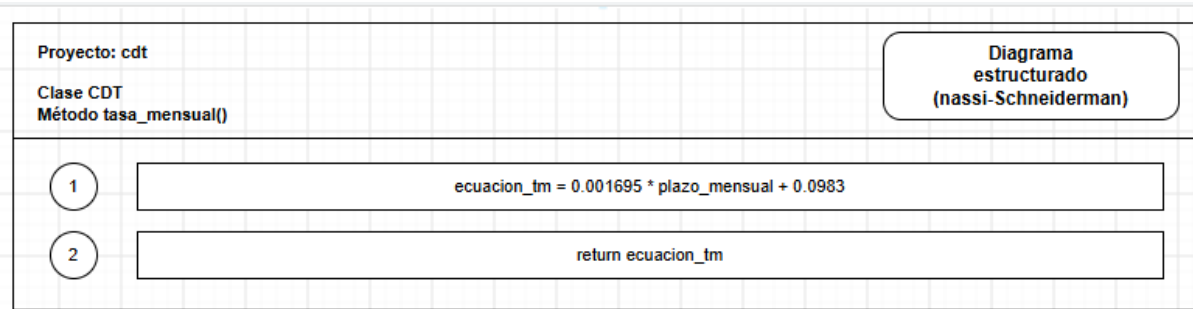
3. Diagramas estructurados (Nassi-Schneiderman)

Clase Presentación

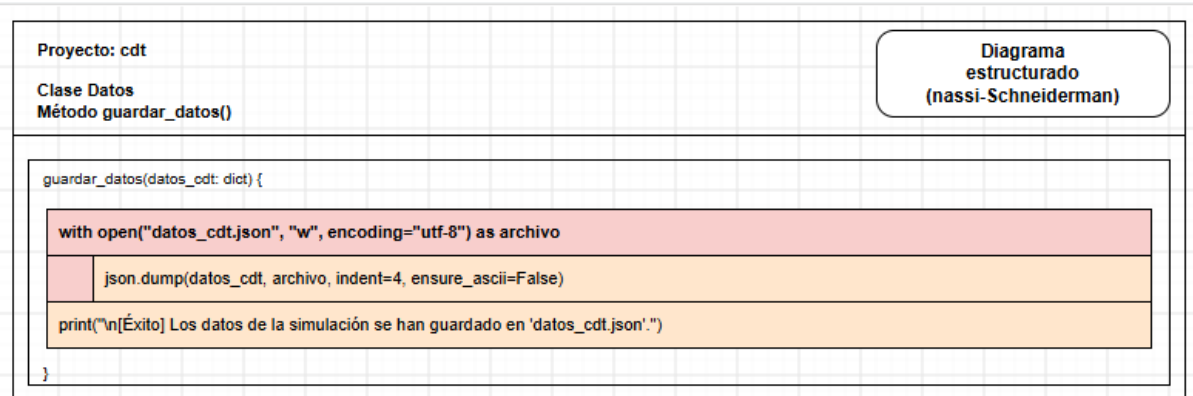




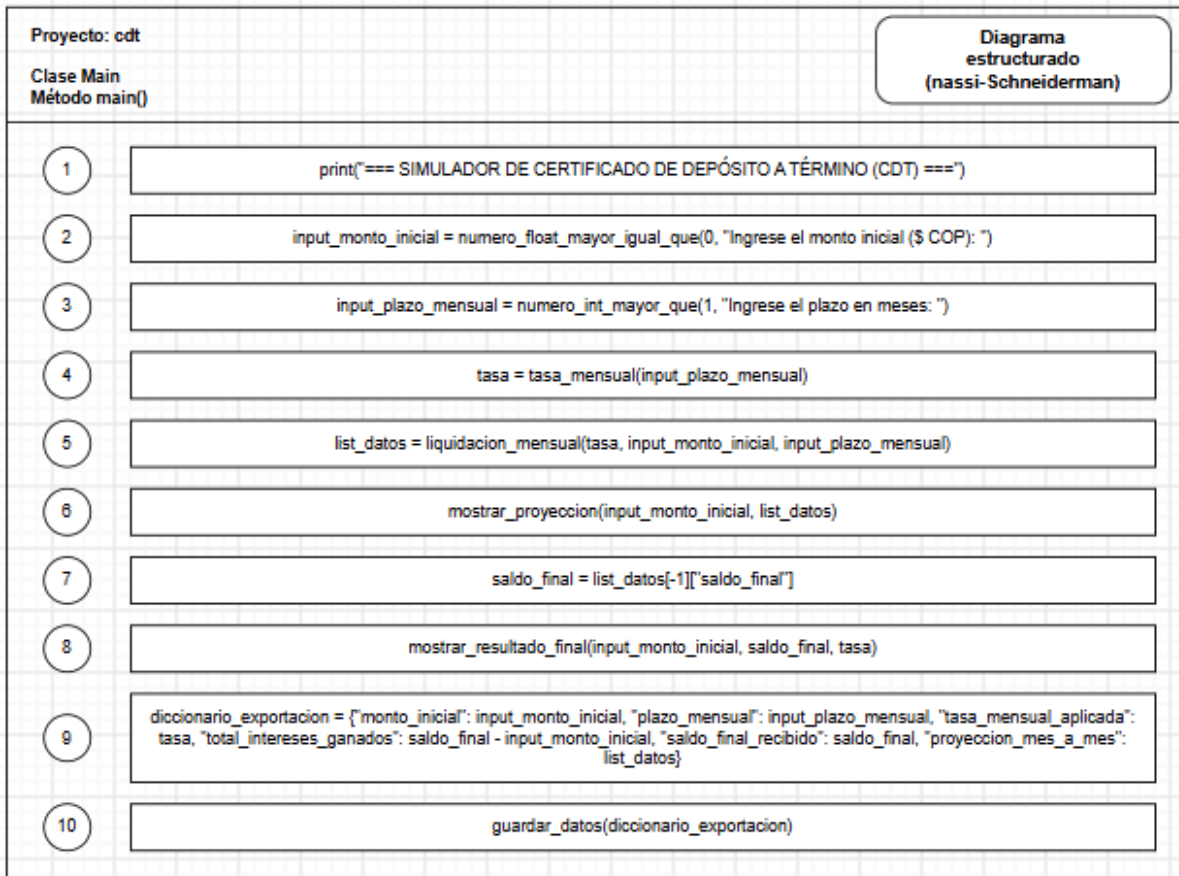
Clase CDT



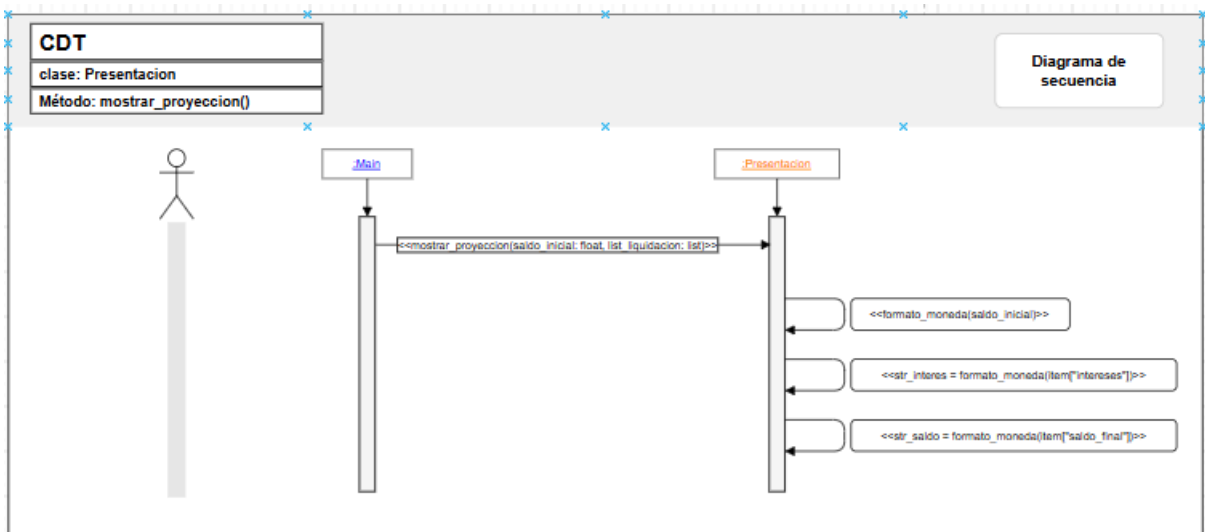
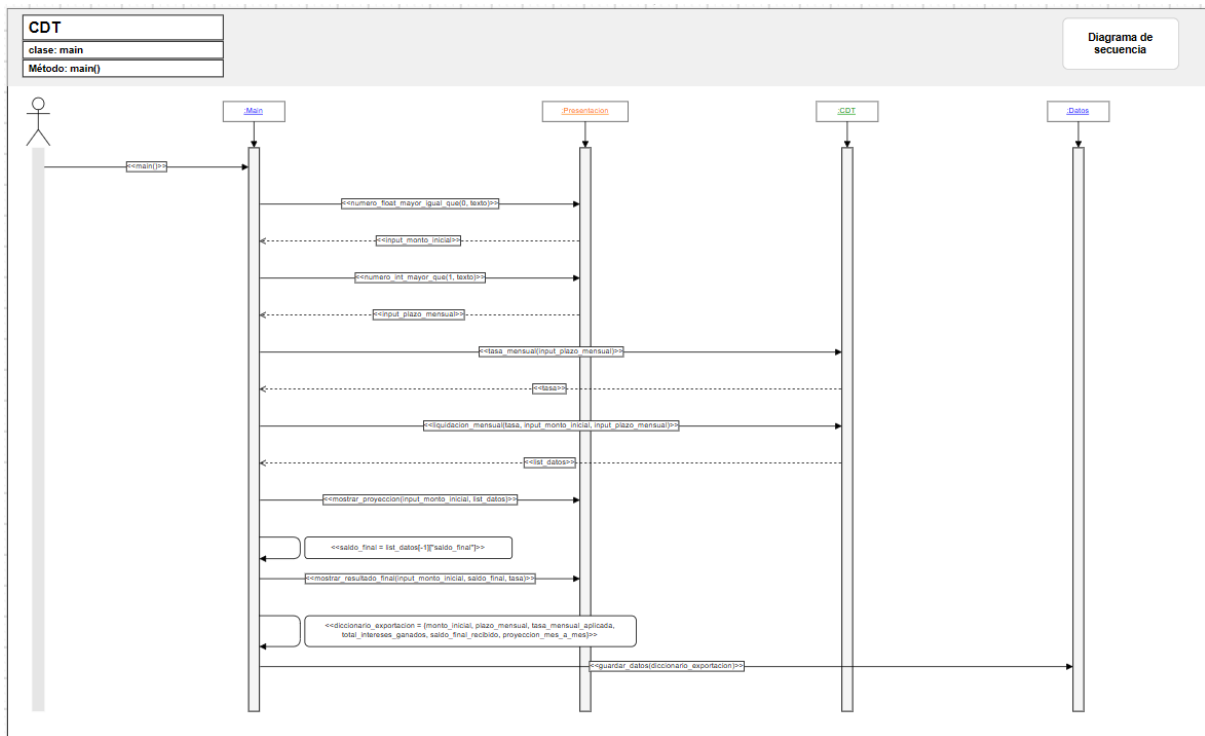
Clase Datos

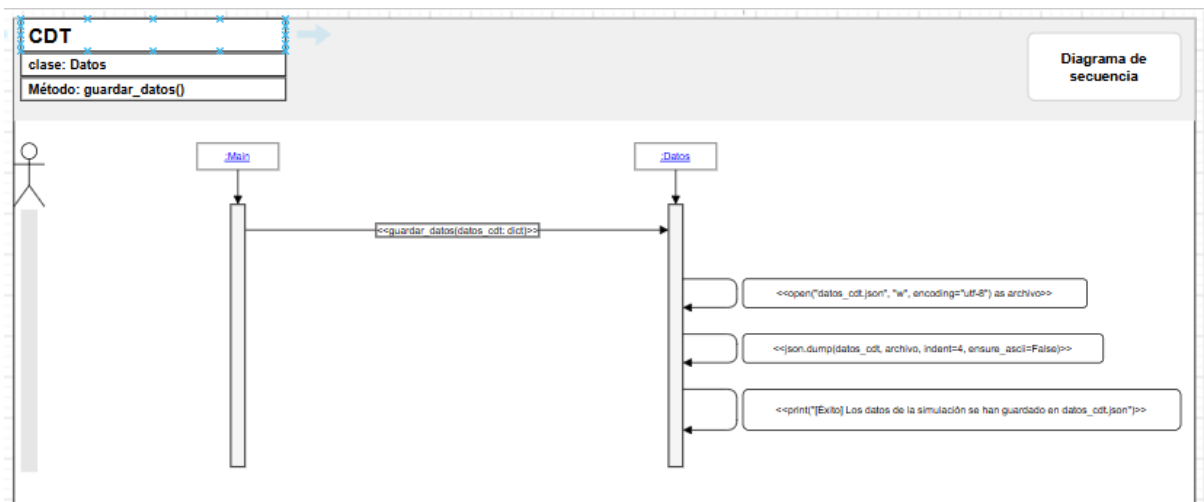
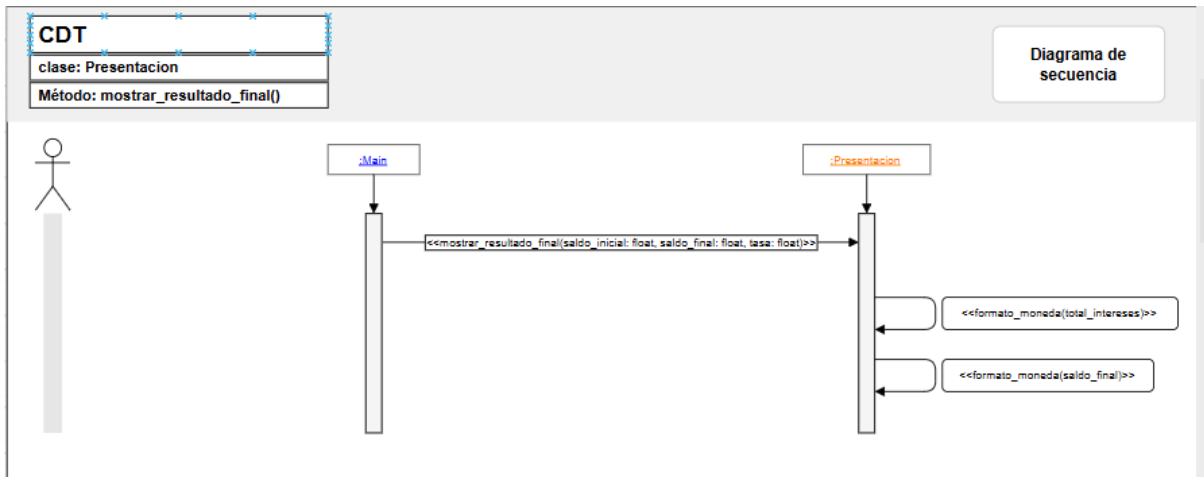


Clase Main



4. Diagramas de secuencia





Código:

```
# Fredy Omar Avila Triana - 01251151005
# Andres Felipe Castilla Caselles - 01251151011
import json
import os

#
=====
# FUNCIONES PARA GUARDAR LOS DATOS
#
=====
def guardar_datos(datos_cdt: dict):
    # Se recomienda el uso de 'with' para asegurar que el archivo se cierre
    automáticamente
    with open("datos_cdt.json", "w", encoding="utf-8") as archivo:
        json.dump(datos_cdt, archivo, indent=4, ensure_ascii=False)
        print("\n[Éxito] Los datos de la simulación se han guardado en
'datos_cdt.json'.")

#
=====
# FUNCIONES DE VALIDACIÓN DE ENTRADA (RF1 y RF2)
#
=====

# Valida y retorna un número flotante estrictamente mayor al número límite
ingresado
def numero_float_mayor_igual_que(n1: float, texto: str) -> float:
    while True:
        try:
            input_numero = float(input(texto))
            if input_numero <= n1:
                print(f"Error: Ha ingresado un número menor o igual a {n1}.
Debe ser mayor a cero.")
                continue
            return input_numero
        except ValueError:
            print("Error: Ha ingresado un valor no numérico.")

# Valida y retorna un número entero mayor o igual al número límite ingresado
def numero_int_mayor_igual_que(n1: int, texto: str) -> int:
    while True:
        try:
            input_numero = int(input(texto))
```

```

        if input_numero < n1:
            print(f"Error: El plazo debe ser un entero mayor o igual a {n1}.")
            continue
        return input_numero
    except ValueError:
        print("Error: Ha ingresado un valor no numérico o decimal.")

#
=====
# FUNCIONES DE CÁLCULO FINANCIERO (RF3 y RF4)
#
=====

# Calcular la tasa_mensual (tm: tasa mensual en porcentaje) a partir del plazo
en meses
def tasa_mensual(plazo_mensual: int) -> float:
    ecuacion_tm = 0.001695 * plazo_mensual + 0.0983
    return ecuacion_tm

# Calcular la liquidacion_mensual retorna en lista para el guardado de datos
# Regla de negocio: Sin redondeos intermedios en los cálculos
def liquidacion_mensual(tasa: float, saldo_inicial: float, plazo_mensual: int)
-> list:
    list_liquidacion = []
    saldo_actual = saldo_inicial

    for mes in range(1, plazo_mensual + 1):
        # Interés mensual = saldo anterior * (tasa / 100)
        interes_mes = saldo_actual * (tasa / 100.0)
        # Saldo nuevo = saldo anterior + interés
        saldo_actual += interes_mes

        # Guardamos los datos de cada mes en un diccionario
        list_liquidacion.append({
            "mes": mes,
            "intereses": interes_mes,
            "saldo_final": saldo_actual
        })

    return list_liquidacion

```

```

#
=====
# FUNCIONES DE FORMATO Y PRESENTACIÓN (RF5 y RF6)
#
=====

# Convierte valores numéricos a formato de moneda COP: $ 1.014.330,07
def formato_moneda(valor: float) -> str:
    # Formateamos con 2 decimales y comas para miles, luego intercambiamos
    caracteres
    cadena_formateada = f"{valor:,.2f}".replace(",", "X").replace(".",
    ",").replace("X", ".")
    return f"$ {cadena_formateada}"

# Proyección de la tabla mes a mes iniciando en el mes 0 (RF5)
def mostrar_proyeccion(saldo_inicial: float, list_liquidacion: list):
    print("\n" + "=" * 60)
    print(f"{'PROYECCIÓN DEL CDT':^60}")
    print("=" * 60)
    print(f"{'Mes':^6} | {'Intereses':^24} | {'Saldo final mes':^24}")
    print("-" * 60)

    # Mes 0 (monto inicial, sin intereses devengados)
    print(f"{0:^6} | {'':^24} | {formato_moneda(saldo_inicial):>24}")

    # Meses del 1 al plazo acordado
    for item in list_liquidacion:
        str_interes = formato_moneda(item["intereses"])
        str_saldo = formato_moneda(item["saldo_final"])
        print(f"{item['mes']:^6} | {str_interes:>24} | {str_saldo:>24}")

    print("=" * 60)

# Mostrar resultados finales y totales devengados (RF6)
def mostrar_resultado_final(saldo_inicial: float, saldo_final: float, tasa:
float):
    # Total intereses = Saldo final menos monto inicial
    total_intereses = saldo_final - saldo_inicial
    tasa_formateada = f"{tasa:.4f}".replace(".", ",")

    print("\n" + "=" * 60)
    print(f"{'RESULTADO FINAL':^60}")
    print("=" * 60)
    print(f"Tasa mensual aplicada : {tasa_formateada}%")

```

```

print(f"Total intereses ganados: {formato_moneda(total_intereses)}")
print(f"Saldo final recibido : {formato_moneda(saldo_final)}")
print("=" * 60)

#
=====
# PROGRAMA PRINCIPAL
#
=====

def main():
    print("=== SIMULADOR DE CERTIFICADO DE DEPÓSITO A TÉRMINO (CDT) ===")

    # Recolección y validación de datos (RF1 y RF2)
    input_monto_inicial = numero_float_mayor_igual_que(0, "Ingrese el monto inicial ($ COP): ")
    input_plazo_mensual = numero_int_mayor_que(1, "Ingrese el plazo en meses: ")

    # Cálculo de la tasa mensual (RF3)
    tasa = tasa_mensual(input_plazo_mensual)

    # Liquidación de intereses mes a mes (RF4)
    list_datos = liquidacion_mensual(tasa, input_monto_inicial, input_plazo_mensual)

    # Mostrar tabla de proyección (RF5)
    mostrar_proyeccion(input_monto_inicial, list_datos)

    # Mostrar resumen final (RF6)
    saldo_final = list_datos[-1]["saldo_final"]
    mostrar_resultado_final(input_monto_inicial, saldo_final, tasa)

    # Estructurar los datos y guardarlos en JSON
    diccionario_exportacion = {
        "monto_inicial": input_monto_inicial,
        "plazo_mensual": input_plazo_mensual,
        "tasa_mensual_aplicada": tasa,
        "total_intereses_ganados": saldo_final - input_monto_inicial,
        "saldo_final_recibido": saldo_final,
        "proyeccion_mes_a_mes": list_datos
    }

    guardar_datos(diccionario_exportacion)

```

```
if __name__ == "__main__":
    main()
```

Prueba Funcionamiento:

```
PS C:\Dev\4to semestre\GBD\Preparcial> & c:\python314\python.exe "c:/Dev/4to semestre/GBD/Preparcial/preparcial_CDT.py"
=== SIMULADOR DE CERTIFICADO DE DEPÓSITO A TÉRMINO (CDT) ===
Ingrese el monto inicial ($ COP): 1000000
Ingrese el plazo en meses: 12

=====
PROYECCIÓN DEL CDT
=====
```

Mes	Intereses	Saldo final mes
0		\$ 1.000.000,00
1	\$ 1.186,40	\$ 1.001.186,40
2	\$ 1.187,81	\$ 1.002.374,21
3	\$ 1.189,22	\$ 1.003.563,42
4	\$ 1.190,63	\$ 1.004.754,05
5	\$ 1.192,04	\$ 1.005.946,09
6	\$ 1.193,45	\$ 1.007.139,55
7	\$ 1.194,87	\$ 1.008.334,42
8	\$ 1.196,29	\$ 1.009.530,70
9	\$ 1.197,71	\$ 1.010.728,41
10	\$ 1.199,13	\$ 1.011.927,54
11	\$ 1.200,55	\$ 1.013.128,09
12	\$ 1.201,98	\$ 1.014.330,07

```
=====
RESULTADO FINAL
=====
Tasa mensual aplicada : 0,1186%
Total intereses ganados: $ 14.330,07
Saldo final recibido  : $ 1.014.330,07
=====

[Éxito] Los datos de la simulación se han guardado en 'datos_cdt.json'.
PS C:\Dev\4to semestre\GBD\Preparcial>
```